



Kubernetes Zero to Hero

- Basics & Architecture
- Pods & Deployments
- Kubernetes Services & Ingress
- K8s custom resources & RBAC

Kubernetes:

"Kubernetes is an open-source orchestration system for managing containerized applications across a cluster of machines. While Docker runs containers, Kubernetes automates deployment, scaling, and management of those containers in production."



Why Not Just Use Docker?

Feature	Docker Alone	Kubernetes
Container creation	☑ Yes	☑ Yes (via Docker/Container Runtime)
Automatic scaling	✗ Manual	☑ Yes (horizontal pod autoscaling)
Load balancing	✗ Needs manual setup	☑ Built-in
Self-healing (auto-restart)	✗ No	☑ Yes
Rolling updates	✗ Needs custom script	☑ Built-in
Multi-container app mgmt	✗ Manual networking & linking	☑ Managed via Pods and Services
Multi-host support	✗ No	☑ Yes (clustered deployment)



Basic Kubernetes Concepts

Component	Description
Pod	The smallest unit in K8s; contains one or more containers
Node	A machine (physical or virtual) where containers run
Cluster	A group of nodes managed by Kubernetes
Deployment	Defines how to run and manage a set of pods (e.g., replicas, updates)
Service	Exposes a set of Pods as a network service (with IP and load balancing)
Ingress	Routes external traffic to Services based on rules (like a reverse proxy)
ConfigMap/Secret	Manage app configuration and sensitive data

Namespace	Virtual clusters inside a K8s cluster for organizing apps
-----------	---

Tools That Work with Kubernetes

Tool	Purpose
kubectl	CLI to interact with Kubernetes
Helm	Package manager for K8s apps
Minikube	Run Kubernetes locally
K3s / Kind	Lightweight K8s distributions
Prometheus	Monitoring
ArgoCD	GitOps (K8s + Git) deployments

Architecture of Kubernetes: -

Kubernetes architecture is divided into 2 main parts

1. Control plane (Master components)
2. Data plane (Worker node components)

The **data plane** is responsible for executing your actions, while the **control plane** is the one that is controlling these actions.

1. Components in the Control Plane (Master Component)

API Server:

- This component acts as the heart and core of Kubernetes.
- It takes all incoming requests from the external world, including user requests to create resources like pods.
- It decides on necessary actions and interacts with other control plane components.
- It basically exposes Kubernetes to the external world. Every request to the cluster is received by the API server.

Scheduler:

- The scheduler is responsible for scheduling pods or other resources onto suitable worker nodes.
- The API server might decide that a resource should be scheduled, and the scheduler acts on that information, determining which specific node (e.g., Node 1 or Node 2) is best suited for the task.

etcd:

- It is a key-value store where all Kubernetes cluster information is stored as objects or key-value pairs.
- Without etcd, there is no cluster-related information, and it is essential for restoring the cluster.

Controller Manager:

- Kubernetes supports features like Auto scaling, which relies on components called controllers (e.g., replica set).
- A replica set controller, for example, ensures that a specified number of pod replicas are always running.
- The Controller Manager is the component that ensures these various inbuilt Kubernetes controllers are always up and running.

CCM (Cloud Controller Manager):

- This component is relevant when running Kubernetes on cloud platforms like AWS (EKS), Azure (AKS), or Google Cloud (GKE).
- It acts as a translator between Kubernetes requests and the underlying cloud provider's API.
- For instance, if a user requests a load balancer or storage, the CCM translates this request into the specific API calls the cloud provider understands.
- This component is often an open-source utility, and cloud providers can implement their specific logic within it to support Kubernetes on their platform.
- If you are running Kubernetes on-premises, this component is generally not required.

1. Components found in the Data Plane (Worker Node)

Cubelet:

- This component is present on each worker node.
- The **cubelet** is responsible for **running your pod** and ensuring that the **pod** is always in a **running state**.
- If a pod is not running, the cubelet informs a component in the control plane that something has gone wrong, so that necessary action can be taken.

kube-proxy:

- This component provides networking for pods and containers.
- It is responsible for tasks like allocating IP addresses to pods/containers and providing default load balancing capabilities.
- It uses IP tables on the Linux machine for networking configuration.
- When you auto-scale a pod to multiple replicas, kube-proxy helps distribute requests between them.
- Every worker node will have this component.

Container Runtime:

- This is the component that actually runs your container.
- Just as Docker needs a container runtime (Docker shim), Kubernetes also needs one because pods contain containers.
- Unlike Docker which has a single support for Docker shim, Kubernetes supports multiple container runtimes like Docker shim, containerd, or CRI-O.
- Kubernetes has a standard called the container interface, and any container runtime that implements this standard can be used with Kubernetes.
- While sometimes not explicitly listed in documentation, container runtime is required on worker nodes.

These three components on the worker node provide the essential functionality to run your application.

Cubelet handles deployment, kube-proxy handles networking, and container runtime provides the execution environment.

Pod

- pod is nothing but the small unit of Kubernetes which can contain one or two containers.
 - Process: - we write the pod.yaml file and we mention all the information over there regarding the image which we want to run on container and container run inside the pod.
 - Below I have posted how to create the pod with our local machine with the help of minikube tool.
 - Follow this:- <https://dev.to/nandkishor/guide-to-install-minikube-on-windows-to-practice-kubernetes-and-create-your-first-pod-14o8>
-

Deployment

- The primary reason to use a Deployment instead of creating a Pod directly is to leverage Kubernetes' built-in auto-healing and auto-scaling behaviors.
Pods themselves do not have this capability

Key Concept: -

- When you create a [Deployment resource](#), it does not directly create the Pods. Instead, a Deployment resource first creates a [Replica Set](#),
- The Replica Set is identified as a Kubernetes controller.
- This [Replica Set](#) is then responsible for [creating the Pods](#).

- The crucial role of the Replica Set created by the Deployment is to implement the auto-healing feature of your Pods and to ensure that the desired number of Pod replicas is always present on the cluster.
-

Here's how the auto-healing and auto-scaling aspects work via the Deployment and Replica Set:

Auto-healing:

- If a Pod created by a Replica Set gets **deleted** (even accidentally by a user) or fails for some reason, the **Replica Set controller** will notice that the actual number of running Pods is less than the desired number specified in the **Deployment YAML file**.
- Because the controller's job is to ensure the **desired state** matches the **actual state**.
- It will automatically **create** a new Pod to bring the count back to the **desired number**.

Auto-scaling:

- If you need to **increase** the number of **Pod replicas** (e.g., due to high load), you can simply modify the **Deployment YAML** file to increase the replica count
 - When you apply the updated configuration or YAML file, the **Replica Set** will detect the change in the **desired state** and automatically create the additional Pods required to reach the **new replica count**. This can be done without disturbing the existing running Pods.
-

Steps to create the Deployment with the help of deploy:(Hands-on)

Step 1: Create the YAML file which can store all the information of deployment like image, replicas, Api version, containers etc.

Eg. DeployDemo.yaml

Step 2: Run the following command to create the deployment.

```
kubectl apply -f DeployDemo.yaml
```

Step 3: check the status of deploy, pods and replica sets these all are got created or not by running below command

```
kubectl get all
```

Step 4: Run the below command to check the in details information of pods

```
Kubectl get pods -w
```

Step 5: Use below command to delete the pod

```
kubectl delete pod <pod name>
```

Kubernetes Services

- Kubernetes Services are a very critical component in the world of Kubernetes.
- In production scenarios, while you typically deploy your application using a deployment, you will most often create a service for that deployment.

Why we need services ?

- The necessity for a service arises from challenges faced when trying to access applications running inside Kubernetes pods directly.
- When a developer or devops engineer **deploys** an application as a **deployment**, Kubernetes creates a **replica set** which in turn creates the specified number of **pod replicas**.
- While Kubernetes offers **auto-healing** capability – meaning if a pod goes down, a new one is created – the IP address of the new pod getting change.
- If **external users** or other applications are accessing the pods directly using **their IP addresses**, these connections will fail when the **pod restarts** and gets a **new IP**. This dynamic nature of pod IP addresses makes applications unreachable even with auto-healing in place.

- To solve this problem services come into the picture there is one function called Service Discovery which help to solve this problem .

This is where Kubernetes Services become essential. A service acts as a stable point of access on top of the deployment and its pods.

Kubernetes Services offer three primary advantages or functions:

1. Load Balancing: (Distribute the load equally on all pods)

- Services distribute incoming traffic across the multiple replicas of your application's pods.
- This is crucial for handling **multiple number of requests**, preventing a **single pod** from getting **overwhelmed** and going down due to excessive load. The service utilizes a component within Kubernetes called **kube-proxy** to perform this load balancing.

2. Service Discovery:

- Services provide a stable way for users or other applications to discover and connect to your application, even as the underlying pod IP addresses change. Instead of tracking ephemeral pod IP addresses, **a service tracks the pods using labels and selectors.**
- When deploying pods (often via a deployment), you apply a **common label** to them, such as ``app: payment``.
- **The service is configured to select pods based on this specific label.**
- Regardless of how many times a pod goes down and is recreated with a **new IP address** by the auto-healing mechanism (replica set), the new pod will be created with the **same label** defined in the original **manifest**.
- The service continuously watches for **pods with this label**. When a new pod with the matching label appears (due to auto-healing or scaling up replicas), the service automatically discovers it. If a pod goes down, the service stops sending traffic to it.

- Kubernetes provides a stable **service name** (e.g., `payments.default.SVC` for a service named 'payments' in the 'default' namespace) and a stable **service IP address**.
- **Users access the application through this table name/IP provided by the service, rather than needing to know the individual, dynamic pod IPs.**

3. **Exposing Applications to the World:**

- By default, applications running in pods inside a Kubernetes cluster are only accessible from within the cluster network. Services allow you to make your application accessible from outside the cluster. This is achieved by specifying different **types** of services.

There are three main default types of Kubernetes Services for exposing applications:

1. ClusterIP: (within cluster)

- This is the **default service type**. A service of this type gets a stable IP address that is only reachable **from within the Kubernetes cluster network**.
- This type primarily provides the Discovery and Load Balancing benefits for internal communication within the cluster. Only users or services that have access to the internal container network can access the application.

1. NodePort: (withing worker node,org)

- This type allows the application to be accessed from **outside the cluster network**, but typically within the **same organization or network** that has access to the **worker nodes**.
- The service opens a static port (**NodePort**) on **each worker node** in the cluster.
- Any traffic sent to the **IP address of any worker node** on this specific NodePort will be forwarded by the service to the appropriate pods.
- Whoever has access to the worker node IP addresses can access the application.

1. LoadBalancer: (internet)

- This type is designed to **expose your application to the external world** (the internet).
- When deployed on a cloud provider (like AWS, GCP, Azure), creating a service of type LoadBalancer automatically provisions an external load balancer provided by the cloud platform (e.g., an Elastic Load Balancer on AWS).
- This cloud load balancer is assigned a **public IP address**. Anyone with access to the **internet** can **access your application** through this **public IP address provided by the cloud load balancer**.
- Note that this service type may not work by default on local Kubernetes clusters like MiniKube; alternatives like Ingress are needed in such cases.

Choosing the right service type depends on your application's accessibility requirements. For example, a globally accessible application like amazon.com would typically use a LoadBalancer type service. An application intended only for internal use within an organization might use NodePort or ClusterIP.

Kubernetes Ingress:

- Kubernetes Ingress provides a centralized and efficient way to manage external access to services running within a Kubernetes cluster. By defining rules for routing incoming traffic based on host, path, or other criteria, Ingress simplifies the process of exposing multiple services through a single entry point.
 - It offers valuable features like load balancing, SSL/TLS termination, name-based virtual hosting, and path-based routing
-

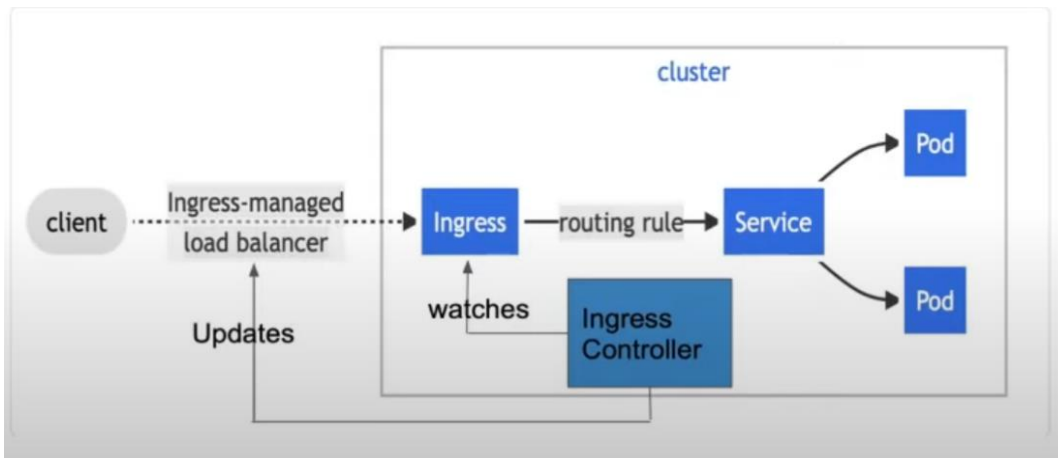
How Does Ingress Work?

Ingress Architecture:

```
[Client] ---> [Ingress Controller] ---> [Service] ---> [Pods]
```

Ingress Rules

1. **Ingress Controller:** A pod running inside the cluster that watches for Ingress resources and routes traffic accordingly (e.g., NGINX, Traefik, HAProxy).
2. **Ingress Resource:** A set of rules defining how to route external requests to services.
3. **Service:** Points to the actual pods (application).



E Ingress Components:

1. Ingress Resource (YAML):
 - a. Contains routing rules (paths, hosts).
 - b. Optionally includes TLS config.
 2. Ingress Controller:
 - a. The actual software that reads the Ingress rules and performs the routing.
 - b. You have to install it separately (e.g., NGINX Ingress Controller).
-

`$ kubectl get pods -A | grep nginx ->` command to check the particular Nginx ingress

why are we not use load balancer service instead of ingress??

Load Balancer vs Ingress in Kubernetes

Feature	Load Balancer Service	Ingress
Purpose	Exposes a single service externally with a dedicated IP.	Routes traffic to multiple services via HTTP/HTTPS.
IP Usage	Each Load Balancer uses a dedicated external IP.	One IP or domain for multiple services.
Routing Logic	No routing logic beyond basic port mapping.	Advanced routing (e.g., path- or host-based).
TLS Support	Manual setup per service.	Centralized TLS via Ingress annotations.
Cost in Cloud	More expensive (each Load Balancer = paid resource).	Cheaper (1 Load Balancer + ingress routes many apps).
Flexibility	Simple, limited features.	Rich features: URL rewriting, SSL, auth, etc.

Why We Use Ingress Instead of Load Balancer in Most Cases

Scalability and Simplicity

- In production, we often have **many services**.
- With Ingress, **one IP/domain can expose all services**.
- Example:
 - app1.example.com → App 1
 - app2.example.com/api → App 2

Cost-Effective

- In cloud (AWS, GCP, Azure), **each LoadBalancer costs money**.
- Ingress uses **only one LoadBalancer**, then internally routes traffic.

Centralized HTTPS / TLS / Auth

- Ingress lets you **configure SSL in one place** using annotations.

- Easy to manage **Let's Encrypt** or other TLS certs.

✓ **Advanced Routing**

- Ingress supports **path-based** and **host-based** routing.
 - /api goes to API service
 - /web goes to frontend
- LoadBalancer doesn't offer this natively.

🔧 **When Should You Use a LoadBalancer?**

- For **quick testing or dev environments**.
- If you're running **just one service** that needs external access.
- If you're not using HTTP/S (e.g., TCP, gRPC on a specific port).
- For **internal apps** where you don't need path/host routing.

🧠 **Summary**

- ✓ Use **LoadBalancer**: Simple, single app, or raw TCP/UDP traffic.
- ✓ Use **Ingress**: Multiple apps, HTTP(S), path/host routing, TLS, or production environments.

Kubernetes RBAC :

RBAC (Role-Based Access Control) is a method for regulating access to Kubernetes resources based on the roles of individual users or service accounts within a cluster.

It allows you to grant or restrict permissions such as:

- Who can create, view, update, or delete resources like pods, deployments, secrets, etc.
- And where (which namespace or cluster-wide).

🤖 **Why Do We Need RBAC?**

Without RBAC, **every user or service account** might have unrestricted access — which is a **major security risk**.

RBAC is used to:

- Enforce **principle of least privilege**
 - Ensure **secure and auditable** access
 - Control what **developers, admins, or automated tools** can do
-

What Are Kubernetes Custom Resources (CRs)?

Ans:-

Custom Resources are **extensions of the Kubernetes API**. They let you **add your own resource types** to a Kubernetes cluster — just like built-in ones (Pod, Deployment, etc.).

📌 Example: You can define a custom resource called Database to manage DBs like MySQL, Postgres as a Kubernetes-native object.

What Is a Custom Controller?

A **Custom Controller** is a **program or operator** that **watches your Custom Resource (CR)** and **acts on it**—for example, it can:

- Create and manage Pods, Services, ConfigMaps, etc.
- Enforce rules or business logic when your CR is created or updated.

Together, CR + Controller = Kubernetes-native automation.

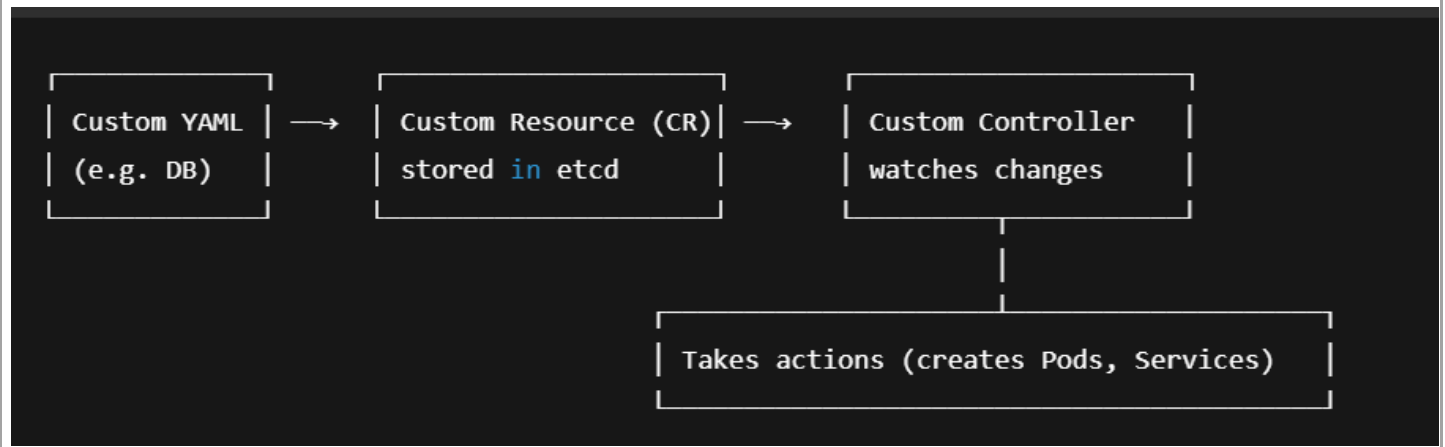
Why Use Custom Resources and Controllers?

Use Case	Example
Manage custom infra	Databases, caches, messaging queues
Extend K8s behavior	Auto-scaling in custom ways
Simplify complex deployments	Managing a multi-step app install as one CR

Create your own abstraction

Define high-level "App" object instead of working with Pod/Service separately

🔗 How It Works (Concept Flow)



🔗 Key Components

1. CRD – CustomResourceDefinition

Defines the schema for your custom resource.

```
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  name: databases.example.com
spec:
  group: example.com
  names:
    kind: Database
    plural: databases
    singular: database
  scope: Namespaced
  versions:
    - name: v1
      served: true
      storage: true
      schema:
        openAPIV3Schema:
          type: object
          properties:
            spec:
              type: object
              properties:
                engine:
```

```
type: string
version:
type: string
```

2. Custom Resource (Instance of the CRD)

```
apiVersion: example.com/v1
kind: Database
metadata:
  name: mydb
spec:
  engine: mysql
  version: "8.0"
```

3. Custom Controller (a Go/Python/JavaScript program)

- Watches for Database resources
- Creates corresponding StatefulSets/Deployments for MySQL
- Deletes them when the Database object is deleted

You can write your controller using:

- **Client-Go** (Go SDK for Kubernetes)
- **Kubebuilder** (easiest way in Go)
- **Operator SDK** (for advanced features)
- **client-python** or JavaScript clients

Real-World Examples

Operator / Project	Description
Cert-Manager	Automates SSL certs using CRs like Certificate, Issuer
Prometheus Operator	Manages Prometheus with CRs like ServiceMonitor
ElasticSearch Operator	Deploy and manage ES clusters using CRs
ArgoCD/Flux	Use CRs like Application to manage GitOps flows

Summary

Term	Description
CRD	Schema/definition of a new resource

CR	An actual object created from the CRD
Controller	Watches CR and performs logic
Operator	CRD + Controller combo used to manage complex systems

Conclusion

Custom Resources + Custom Controllers = Powerful K8s Extension.

You can manage anything as code in Kubernetes:

- Databases
- Cloud resources
- Complex apps

HAPPY LEARNING!!

Author: Nandkishor Khandare

LinkedIn: <https://www.linkedin.com/in/nandkishor-khandare-616492215/>

GitHub: <https://github.com/I-am-nk>
